Resolving OpenAlex HTTP 429 Errors

https://gemini.google.com/app/1dbf0df115305e70

User prompt: """ CODE A harvester/openalex.py - Asynchronous OpenAlex Harvester with cursor-based pagination. Unified Academic Paper Harvester, Verifier & Notification Pipeline. """ import aiohttp from typing import List, Tuple, Optional, Dict, Any from config import cfg from harvester.models import Paper, sanitize_doi from harvester.base import BaseAsyncHarvester class OpenAlexHarvester(BaseAsyncHarvester): """Harvests academic papers from OpenAlex with cursor-based deep pagination.""" def __init__(self): super().__init__("OpenAlex", rate_limit_per_second=cfg.OPENALEX_RATE_LIMIT) self.base_url = "https://api.openalex.org/works" @staticmethod def _reconstruct_abstract(inverted_index: Optional[Dict[str, List[int]]]) -> str: """Reconstructs text from OpenAlex's abstract_inverted_index.""" if not inverted_index: return "" positions = {} for word, idxs in inverted_index.items(): for idx in idxs: positions[idx] = word sorted_words = [positions[i] for i in sorted(positions.keys())] return " ".join(sorted_words) async def fetch_page(self, session: aiohttp.ClientSession, query: str, page_token: Optional[str] = None, page_size: int = 50, year_range: Optional[str] = None, doc_filter: Optional[str] = None) -> Tuple[List[Paper], Optional[str], bool]: cursor = page_token or "" params = { "search": query, "per_page": min(page_size, 100), "cursor": cursor } # Build OpenAlex filter string filters = [] if year_range and year_range.lower() != "all years": # Check for pattern like "2021-2026" or single year parts = [p.strip() for p in year_range.split("-") if p.strip().isdigit()] if len(parts) == 2: filters.append(f"from_publication_date:{parts[0]}-01-01,to_publication_date:{parts[1]}-12-31") elif len(parts) == 1: filters.append(f"publication_year:{parts[0]}") if doc_filter: doc_lower = doc_filter.lower() if "journal" in doc_lower: filters.append("type:article") elif "conference" in doc_lower: filters.append("type:proceedings-article") elif "preprint" in doc_lower: filters.append("type:preprint") if filters: params["filter"] = ",".join(filters) data = await self.get_json(session, self.base_url, params=params) if not data: return [], None, False meta = data.get("meta", {}) next_cursor = meta.get("next_cursor") results = data.get("results", []) papers: List[Paper] = [] for item in results: title = item.get("display_name") or item.get("title") or "" if not title: continue # Authors authors_list = [] for authorship in item.get("authorships", []): author = authorship.get("author", {}) name = author.get("display_name") if name: authors_list.append(name) authors_str = "; ".join(authors_list) # Year year_str = str(item.get("publication_year") or "") # Venue & Publisher primary_loc = item.get("primary_location") or {} source_obj = primary_loc.get("source") or {} venue = source_obj.get("display_name") or "" issns = source_obj.get("issn") or [] # DOI & Landing URL doi = sanitize_doi(item.get("doi") or "") landing_url = primary_loc.get("landing_page_url") or (f"https://doi.org/{doi}" if doi else "") # PDF link pdf_link = (item.get("open_access") or {}).get("oa_url") or primary_loc.get("pdf_url") or "" # Publication Type raw_type = item.get("type", "article") pub_type = "Journal Article" if raw_type in ("proceedings-article", "proceedings"): pub_type = "Conference Paper" elif raw_type in ("book-chapter", "book"): pub_type = "Book Chapter" elif raw_type == "preprint": pub_type = "Preprint" # Open Access status is_oa = (item.get("open_access") or {}).get("is_oa") is True # Abstract abstract = self._reconstruct_abstract(item.get("abstract_inverted_index")) paper = Paper(title=title, authors=authors_str, year=year_str, publication_type=pub_type, venue=venue, doi=doi, url=landing_url, pdf_link=pdf_link, source="OpenAlex", abstract=abstract, access_type="Open Access" if is_oa else "Closed", issn_list=issns, open_access=is_oa) papers.append(paper) has_more = bool(next_cursor and len(results) > 0) return papers, next_cursor, has_more ---------- """ CODE B services/academic/openalex.py - OpenAlex REST API Client. Unified Academic Paper Harvester, Verifier & Notification Pipeline. Requirements: - Enforce OpenAlex rate limits (10 req/s). - Include mailto: {email} User-Agent etiquette and optional api_key header. - Circuit breaker protection and exponential backoff retry on HTTP 429. """ import urllib.parse import logging from typing import Optional, Dict, Any, List import requests from config import cfg from utils.rate_limiters import TokenBucketRateLimiter from utils.retry_handlers import execute_with_429_retry from utils.circuit_breaker import CircuitBreaker logger = logging.getLogger("OpenAlexClient") _OPENALEX_RATE_LIMITER = TokenBucketRateLimiter(rate_limit_per_second=cfg.OPENALEX_RATE_LIMIT, safety_buffer_seconds=0.02) class OpenAlexClient: """Client for OpenAlex REST API.""" BASE_URL = "https://api.openalex.org" def __init__(self, circuit_breaker: Optional[CircuitBreaker] = None): self.circuit_breaker = circuit_breaker or CircuitBreaker("OpenAlex API", failure_threshold=3) self.session = requests.Session() self.limiter = _OPENALEX_RATE_LIMITER def get_headers(self) -> Dict[str, str]: headers = { "User-Agent": f"AcademicHarvester/4.0 (mailto:{cfg.POLITE_EMAIL})", "Accept": "application/json" } if cfg.OPENALEX_API_KEY: headers["api_key"] = cfg.OPENALEX_API_KEY return headers def verify_doi(self, doi: str) -> Optional[Dict[str, Any]]: """Verifies metadata for a DOI against OpenAlex.""" if not doi or self.circuit_breaker.is_tripped(): return None clean_doi = doi.strip() url = f"{self.BASE_URL}/works/https://doi.org/{urllib.parse.quote(clean_doi)}" self.limiter.acquire() def _do_request(): return self.session.get(url, headers=self.get_headers(), timeout=cfg.DEFAULT_TIMEOUT) try: resp = execute_with_429_retry(request_fn=_do_request, max_retries=cfg.MAX_RETRIES, service_name="OpenAlex") if resp is None: self.circuit_breaker.record_failure(None, "Connection timeout") return None if resp.status_code == 200: self.circuit_breaker.record_success() data = resp.json() location = data.get("primary_location") or {} source = location.get("source") or {} is_oa = bool(data.get("open_access", {}).get("is_oa", False)) pdf_url = data.get("open_access", {}).get("oa_url") or "" return { "confirmed": True, "title": data.get("title", ""), "venue": source.get("display_name", ""), "issn_list": source.get("issn", []), "year": str(data.get("publication_year") or ""), "is_oa": is_oa, "pdf_link": pdf_url, "type": data.get("type", "") } elif resp.status_code == 429: self.circuit_breaker.record_failure(429, "Rate limit exceeded") return None elif resp.status_code in (401, 403): self.circuit_breaker.record_failure(resp.status_code, "Auth error") return None return None except Exception as e: logger.warning(f"[OpenAlex] verify_doi failed for '{doi}': {e}") self.circuit_breaker.record_failure(None, str(e)) return None def verify_title(self, title: str) -> Optional[Dict[str, Any]]: """Verifies metadata for a title against OpenAlex Works API.""" if not title or self.circuit_breaker.is_tripped(): return None clean_title = title.strip() url = f"{self.BASE_URL}/works?filter=title.search:{urllib.parse.quote(clean_title)}&per-page=1" self.limiter.acquire() def _do_request(): return self.session.get(url, headers=self.get_headers(), timeout=cfg.DEFAULT_TIMEOUT) try: resp = execute_with_429_retry(request_fn=_do_request, max_retries=cfg.MAX_RETRIES, service_name="OpenAlex Title") if resp is None or resp.status_code != 200: if resp is None: self.circuit_breaker.record_failure(None, "Connection timeout") return None results = resp.json().get("results", []) if not results: return None data = results[0] matched_title = data.get("title", "") from harvester.models import clean_matching_title from difflib import SequenceMatcher s1 = clean_matching_title(clean_title) s2 = clean_matching_title(matched_title) if SequenceMatcher(None, s1, s2).ratio() < 0.65: return None self.circuit_breaker.record_success() location = data.get("primary_location") or {} source = location.get("source") or {} is_oa = bool(data.get("open_access", {}).get("is_oa", False)) pdf_url = data.get("open_access", {}).get("oa_url") or "" raw_doi = data.get("doi", "") or "" doi = raw_doi if doi.startswith("https://doi.org/"): doi = doi[len("https://doi.org/"):] return { "confirmed": True, "title": matched_title, "venue": source.get("display_name", ""), "issn_list": source.get("issn", []), "year": str(data.get("publication_year") or ""), "doi": doi, "is_oa": is_oa, "pdf_link": pdf_url, "type": data.get("type", "") } except Exception as e: logger.warning(f"[OpenAlex] verify_title failed for '{title}': {e}") self.circuit_breaker.record_failure(None, str(e)) return None WHy am i facing 429 ??*

Response: The HTTP 429 ("Too Many Requests") errors are occurring due to rate-limiting mismatches and missing authentication/identification between **Code A** and **Code B**.

1. Code A Completely Bypasses Identification and Polite Pool

OpenAlex requires identification to place requests into their polite pool (10 req/s) or premium pool. Anonymous requests are heavily throttled and quickly trigger HTTP 429.

- **Missing User-Agent/Mailto in Code A:** In `OpenAlexHarvester.fetch_page()`, requests are sent via `session` without setting the OpenAlex polite email or user agent:

  ```
  # Code B sets this header, but Code A does not!
  "User-Agent": f"AcademicHarvester/4.0 (mailto:{cfg.POLITE_EMAIL})"
  ```

  OpenAlex explicitly requires a `mailto:` address in the `User-Agent` header or as a `?mailto=` query parameter. Without it, you are treated as an unauthenticated, low-priority client with strict IP-level rate limits.

- **Missing API Key in Code A:** `cfg.OPENALEX_API_KEY` is completely unused in `OpenAlexHarvester`. If your organization upgraded to an API key, Code A never passes it via headers or query parameters (`?api_key=...`).

2. Harvester and Verifier Do Not Share a Rate Limiter

If `OpenAlexHarvester` (Code A) and `OpenAlexClient` (Code B) run concurrently, they step on each other:

- `OpenAlexClient` uses an internal token bucket:

  ```
  _OPENALEX_RATE_LIMITER = TokenBucketRateLimiter(rate_limit_per_second=cfg.OPENALEX_RATE_LIMIT, ...)
  ```

- `OpenAlexHarvester` relies on `BaseAsyncHarvester(..., rate_limit_per_second=cfg.OPENALEX_RATE_LIMIT)`.

- Because these limiters are isolated instances (one async, one synchronous), if Code A harvests papers while Code B concurrently verifies titles/DOIs from the same egress IP address, the combined request rate easily doubles your allowed limit.

3. OpenAlex Global Daily Budget Limits

OpenAlex applies two distinct layers of rate limiting:

1. **Per-second rate:** 10 requests per second (polite pool with `mailto:`).
2. **Daily volume cap:** Free/polite pool access has a hard limit of **100,000 requests per day per IP/email**.

Once the 100,000-request daily cap is exceeded, OpenAlex returns persistent `429 Too Many Requests` responses for the remainder of the rolling 24-hour window, regardless of whether your per-second limiter waits properly.

4. Query Parameter Syntax: `per_page` vs `per-page`

- In `OpenAlexHarvester` (Code A):

  ```
  params = { "per_page": min(page_size, 100), ... }
  ```

- In `OpenAlexClient` (Code B):

  ```
  url = f"...&per-page=1"
  ```

OpenAlex's standard parameter is **`per-page`** (with a hyphen). Passing `per_page` in Code A falls back to OpenAlex's default page size (25 items) instead of 50–100, forcing your crawler to make **2× to 4× more API calls** to fetch the same dataset, consuming your daily quota much faster.

Recommended Fixes

1. **Add Polite Identification / API Key to Code A:** Add headers to `self.get_json()` in `OpenAlexHarvester`:

   ```
   headers = {
       "User-Agent": f"AcademicHarvester/4.0 (mailto:{cfg.POLITE_EMAIL})",
   }
   if getattr(cfg, "OPENALEX_API_KEY", None):
       headers["api_key"] = cfg.OPENALEX_API_KEY

   data = await self.get_json(session, self.base_url, params=params, headers=headers)
   ```

   Or pass `mailto` directly in `params`:

   ```
   params["mailto"] = cfg.POLITE_EMAIL
   ```

2. **Inspect Response Headers in Retry Handlers:** OpenAlex returns rate-limiting telemetry in its headers:
   - Look for `x-ratelimit-limit` and `retry-after` in your 429 response.
   - If `retry-after` is in thousands of seconds, you have tripped the **daily quota** (100k requests/day), not just the per-second rate limiter.
3. **Coordinate the Rate Limiters:** If running multi-process or mixed sync/async pipelines on the same machine, synchronize requests across workers using a centralized Redis token bucket or throttle both components down (e.g., `rate_limit_per_second = 4` instead of `10`).

User prompt: can u clean code A and after that can u write the full code for me so that i can copy paste it ??

Response: Here is the fully cleaned and corrected version of **Code A** (`harvester/openalex.py`).

What was fixed:

1. **Polite Pool Access:** Added the `User-Agent` header with your `mailto:` address, and injected `mailto` and `api_key` directly into the query parameters. This ensures OpenAlex identifies your harvester and places it in the 10 req/s polite pool instead of the severely throttled anonymous pool.
2. **Pagination Parameter:** Changed the incorrect `per_page` to OpenAlex's required `per-page`. This prevents the API from defaulting to 25 items per request, meaning you will make fewer API calls to get the same amount of data, saving your daily quota.

```
"""
harvester/openalex.py - Asynchronous OpenAlex Harvester with cursor-based pagination.
Unified Academic Paper Harvester, Verifier & Notification Pipeline.
"""

import aiohttp
from typing import List, Tuple, Optional, Dict, Any

from config import cfg
from harvester.models import Paper, sanitize_doi
from harvester.base import BaseAsyncHarvester


class OpenAlexHarvester(BaseAsyncHarvester):
    """Harvests academic papers from OpenAlex with cursor-based deep pagination."""

    def __init__(self):
        super().__init__("OpenAlex", rate_limit_per_second=getattr(cfg, 'OPENALEX_RATE_LIMIT', 10))
        self.base_url = "https://api.openalex.org/works"

    @staticmethod
    def _reconstruct_abstract(inverted_index: Optional[Dict[str, List[int]]]) -> str:
        """Reconstructs text from OpenAlex's abstract_inverted_index."""
        if not inverted_index:
            return ""
        positions = {}
        for word, idxs in inverted_index.items():
            for idx in idxs:
                positions[idx] = word
        sorted_words = [positions[i] for i in sorted(positions.keys())]
        return " ".join(sorted_words)

    async def fetch_page(
        self,
        session: aiohttp.ClientSession,
        query: str,
        page_token: Optional[str] = None,
        page_size: int = 50,
        year_range: Optional[str] = None,
        doc_filter: Optional[str] = None
    ) -> Tuple[List[Paper], Optional[str], bool]:
        cursor = page_token or "*"

        # FIX: Changed 'per_page' to 'per-page'
        params = {
            "search": query,
            "per-page": min(page_size, 100),
            "cursor": cursor
        }

        # FIX: Explicitly add Authentication and Polite Pool params
        # (Fallback in case get_json doesn't pass headers properly)
        if hasattr(cfg, 'POLITE_EMAIL') and cfg.POLITE_EMAIL:
            params["mailto"] = cfg.POLITE_EMAIL
        if hasattr(cfg, 'OPENALEX_API_KEY') and cfg.OPENALEX_API_KEY:
            params["api_key"] = cfg.OPENALEX_API_KEY

        # Build OpenAlex filter string
        filters = []
        if year_range and year_range.lower() != "all years":
            # Check for pattern like "2021-2026" or single year
            parts = [p.strip() for p in year_range.split("-") if p.strip().isdigit()]
            if len(parts) == 2:
                filters.append(f"from_publication_date:{parts[0]}-01-01,to_publication_date:{parts[1]}-12-31")
            elif len(parts) == 1:
                filters.append(f"publication_year:{parts[0]}")

        if doc_filter:
            doc_lower = doc_filter.lower()
```

```
            if "journal" in doc_lower:
                filters.append("type:article")
            elif "conference" in doc_lower:
                filters.append("type:proceedings-article")
            elif "preprint" in doc_lower:
                filters.append("type:preprint")

        if filters:
            params["filter"] = ",".join(filters)

        # FIX: Include Polite Pool User-Agent headers
        headers = {
            "User-Agent": f"AcademicHarvester/4.0 (mailto:{getattr(cfg, 'POLITE_EMAIL', 'unknown@example.com')})",
            "Accept": "application/json"
        }
        if hasattr(cfg, 'OPENALEX_API_KEY') and cfg.OPENALEX_API_KEY:
            headers["api_key"] = cfg.OPENALEX_API_KEY

        # Fetch data (passing both params and headers)
        data = await self.get_json(session, self.base_url, params=params, headers=headers)
        if not data:
            return [], None, False

        meta = data.get("meta", {})
        next_cursor = meta.get("next_cursor")
        results = data.get("results", [])

        papers: List[Paper] = []
        for item in results:
            title = item.get("display_name") or item.get("title") or ""
            if not title:
                continue

            # Authors
            authors_list = []
            for authorship in item.get("authorships", []):
                author = authorship.get("author", {})
                name = author.get("display_name")
                if name:
                    authors_list.append(name)
            authors_str = "; ".join(authors_list)

            # Year
            year_str = str(item.get("publication_year") or "")

            # Venue & Publisher
            primary_loc = item.get("primary_location") or {}
            source_obj = primary_loc.get("source") or {}
            venue = source_obj.get("display_name") or ""
            issns = source_obj.get("issn") or []

            # DOI & Landing URL
            doi = sanitize_doi(item.get("doi") or "")
            landing_url = primary_loc.get("landing_page_url") or (f"https://doi.org/{doi}" if doi else "")

            # PDF link
            pdf_link = (item.get("open_access") or {}).get("oa_url") or primary_loc.get("pdf_url") or ""

            # Publication Type
            raw_type = item.get("type", "article")
            pub_type = "Journal Article"
            if raw_type in ("proceedings-article", "proceedings"):
                pub_type = "Conference Paper"
            elif raw_type in ("book-chapter", "book"):
                pub_type = "Book Chapter"
            elif raw_type == "preprint":
                pub_type = "Preprint"

            # Open Access status
            is_oa = (item.get("open_access") or {}).get("is_oa") is True

            # Abstract
            abstract = self._reconstruct_abstract(item.get("abstract_inverted_index"))

            paper = Paper(
                title=title,
                authors=authors_str,
                year=year_str,
                publication_type=pub_type,
```

```
                venue=venue,
                doi=doi,
                url=landing_url,
                pdf_link=pdf_link,
                source="OpenAlex",
                abstract=abstract,
                access_type="Open Access" if is_oa else "Closed",
                issn_list=issns,
                open_access=is_oa
            )
            papers.append(paper)

        has_more = bool(next_cursor and len(results) > 0)
        return papers, next_cursor, has_more
```

User prompt: Can U do the Same for Code B

Response: Here is the cleaned and corrected version of Code B. It is fully ready for you to copy and paste.

I have ensured that the URL parsing is robust, the headers are correctly formatted for the polite pool, and the response parsing handles missing data gracefully without throwing `KeyError` exceptions.

```
"""
services/academic/openalex.py - OpenAlex REST API Client.
Unified Academic Paper Harvester, Verifier & Notification Pipeline.

Requirements:
- Enforce OpenAlex rate limits (10 req/s).
- Include mailto:{email} User-Agent etiquette and optional api_key header.
- Circuit breaker protection and exponential backoff retry on HTTP 429.
"""

import urllib.parse
import logging
from typing import Optional, Dict, Any
import requests

from config import cfg
from utils.rate_limiters import TokenBucketRateLimiter
from utils.retry_handlers import execute_with_429_retry
from utils.circuit_breaker import CircuitBreaker

logger = logging.getLogger("OpenAlexClient")

# Note: If Code A and Code B run in the same process, consider passing a single
# shared TokenBucketRateLimiter to both to strictly enforce the 10 req/s global limit.
_OPENALEX_RATE_LIMITER = TokenBucketRateLimiter(
    rate_limit_per_second=cfg.OPENALEX_RATE_LIMIT,
    safety_buffer_seconds=0.02
)

class OpenAlexClient:
    """Client for OpenAlex REST API."""

    BASE_URL = "https://api.openalex.org"

    def __init__(self, circuit_breaker: Optional[CircuitBreaker] = None):
        self.circuit_breaker = circuit_breaker or CircuitBreaker("OpenAlex API", failure_threshold=3)
        self.session = requests.Session()
        self.limiter = _OPENALEX_RATE_LIMITER

    def get_headers(self) -> Dict[str, str]:
        """Construct headers to ensure inclusion in the OpenAlex polite pool."""
        headers = {
            "User-Agent": f"AcademicHarvester/4.0 (mailto:{cfg.POLITE_EMAIL})",
            "Accept": "application/json"
        }
        if getattr(cfg, "OPENALEX_API_KEY", None):
            headers["api_key"] = cfg.OPENALEX_API_KEY
        return headers

    def verify_doi(self, doi: str) -> Optional[Dict[str, Any]]:
        """Verifies metadata for a DOI against OpenAlex."""
        if not doi or self.circuit_breaker.is_tripped():
            return None

        clean_doi = doi.strip()
        # Remove any existing https://doi.org/ prefix to avoid double-prefixing
```

```
        if clean_doi.startswith(("http://doi.org/", "https://doi.org/")):
            clean_doi = clean_doi.split("doi.org/")[-1]

        url = f"{self.BASE_URL}/works/https://doi.org/{urllib.parse.quote(clean_doi, safe='/')}"

        self.limiter.acquire()

        def _do_request():
            return self.session.get(url, headers=self.get_headers(), timeout=getattr(cfg, 'DEFAULT_TIMEOUT', 10))

        try:
            resp = execute_with_429_retry(
                request_fn=_do_request,
                max_retries=getattr(cfg, 'MAX_RETRIES', 3),
                service_name="OpenAlex"
            )

            if resp is None:
                self.circuit_breaker.record_failure(None, "Connection timeout or max retries exceeded")
                return None

            if resp.status_code == 200:
                self.circuit_breaker.record_success()
                data = resp.json()
                location = data.get("primary_location") or {}
                source = location.get("source") or {}

                open_access_data = data.get("open_access") or {}
                is_oa = bool(open_access_data.get("is_oa", False))
                pdf_url = open_access_data.get("oa_url") or ""

                return {
                    "confirmed": True,
                    "title": data.get("title", ""),
                    "venue": source.get("display_name", ""),
                    "issn_list": source.get("issn", []),
                    "year": str(data.get("publication_year") or ""),
                    "is_oa": is_oa,
                    "pdf_link": pdf_url,
                    "type": data.get("type", "")
                }
            elif resp.status_code == 429:
                self.circuit_breaker.record_failure(429, "Rate limit exceeded")
                return None
            elif resp.status_code in (401, 403):
                self.circuit_breaker.record_failure(resp.status_code, "Authentication/Authorization error")
                return None
            elif resp.status_code == 404:
                # DOI not found in OpenAlex; not a circuit breaker failure, just missing data
                return None

            return None

        except Exception as e:
            logger.warning(f"[OpenAlex] verify_doi failed for '{doi}': {e}")
            self.circuit_breaker.record_failure(None, str(e))
            return None

    def verify_title(self, title: str) -> Optional[Dict[str, Any]]:
        """Verifies metadata for a title against OpenAlex Works API."""
        if not title or self.circuit_breaker.is_tripped():
            return None

        clean_title = title.strip()
        url = f"{self.BASE_URL}/works?filter=title.search:{urllib.parse.quote(clean_title)}&per-page=1"

        self.limiter.acquire()

        def _do_request():
            return self.session.get(url, headers=self.get_headers(), timeout=getattr(cfg, 'DEFAULT_TIMEOUT', 10))

        try:
            resp = execute_with_429_retry(
                request_fn=_do_request,
                max_retries=getattr(cfg, 'MAX_RETRIES', 3),
                service_name="OpenAlex Title"
            )

            if resp is None or resp.status_code != 200:
```

```
            if resp is None:
                self.circuit_breaker.record_failure(None, "Connection timeout or max retries exceeded")
            elif resp.status_code == 429:
                self.circuit_breaker.record_failure(429, "Rate limit exceeded")
            return None

        results = resp.json().get("results", [])
        if not results:
            return None

        data = results[0]
        matched_title = data.get("title", "")

        # Fuzzy match to ensure the search result actually matches the requested title
        from harvester.models import clean_matching_title
        from difflib import SequenceMatcher

        s1 = clean_matching_title(clean_title)
        s2 = clean_matching_title(matched_title)
        if SequenceMatcher(None, s1, s2).ratio() < 0.65:
            return None

        self.circuit_breaker.record_success()
        location = data.get("primary_location") or {}
        source = location.get("source") or {}

        open_access_data = data.get("open_access") or {}
        is_oa = bool(open_access_data.get("is_oa", False))
        pdf_url = open_access_data.get("oa_url") or ""

        raw_doi = data.get("doi", "") or ""
        doi = raw_doi
        if doi.startswith("https://doi.org/"):
            doi = doi[len("https://doi.org/"):]

        return {
            "confirmed": True,
            "title": matched_title,
            "venue": source.get("display_name", ""),
            "issn_list": source.get("issn", []),
            "year": str(data.get("publication_year") or ""),
            "doi": doi,
            "is_oa": is_oa,
            "pdf_link": pdf_url,
            "type": data.get("type", "")
        }

    except Exception as e:
        logger.warning(f"[OpenAlex] verify_title failed for '{title}': {e}")
        self.circuit_breaker.record_failure(None, str(e))
        return None
```

User prompt: """ services/academic/semantic_scholar.py - Semantic Scholar Graph API Client with Strict Rate Limiting. Unified Academic Paper Harvester, Verifier & Notification Pipeline. Requirements: - Strict rate limit of 1 request per second (cumulative across all endpoints) with 1.05s safety buffer. - Exponential backoff retry handler catching HTTP 429 (Too Many Requests). - Outbound headers must include 'x-api-key': os.getenv('S2_API_KEY') on every outbound request. - Token pooling and rotation when multiple keys are provided. """ import os import time import urllib.parse import logging from typing import Optional, Dict, Any, List, Tuple from difflib import SequenceMatcher import requests from config import cfg from utils.rate_limiters import TokenBucketRateLimiter, GLOBAL_SEMANTIC_SCHOLAR_LIMITER from utils.retry_handlers import execute_with_429_retry from utils.circuit_breaker import CircuitBreaker from harvester.models import clean_matching_title, sanitize_doi logger = logging.getLogger("SemanticScholarClient") # Global singleton rate limiter across all Semantic Scholar endpoints: # Strict 1 request per second with 0.05s safety buffer (1.05s total interval) _S2_RATE_LIMITER = GLOBAL_SEMANTIC_SCHOLAR_LIMITER class SemanticScholarClient: """ Client for Semantic Scholar Graph API. Enforces strict 1 req/sec pacing, HTTP 429 exponential backoff, and mandatory 'x-api-key' header forwarding with token rotation. """ BASE_URL = "https://api.semanticscholar.org/graph/v1" def __init__(self, circuit_breaker: Optional[CircuitBreaker] = None): self.circuit_breaker = circuit_breaker or CircuitBreaker("Semantic Scholar API", failure_threshold=3) self.session = requests.Session() self.limiter = _S2_RATE_LIMITER self.key_pool: List[str] = cfg.get_key_pool("semantic_scholar") self.current_key_idx: int = 0 self.last_status_code: Optional[int] = None self.on_429_callback: Optional[Any] = None def _get_api_key(self) -> str: """Retrieves active S2 API key from key pool first, falling back to env/config.""" if self.key_pool: return self.key_pool[self.current_key_idx % len(self.key_pool)].strip() env_key = (os.getenv("S2_API_KEY") or getattr(cfg, "S2_API_KEY", None) or getattr(cfg, "SEMANTIC_SCHOLAR_API_KEY", None) or "") return env_key.strip() def _rotate_key(self): """Rotates to next key in pool upon receiving rate limit or auth response.""" if self.key_pool and len(self.key_pool) > 1: self.current_key_idx = (self.current_key_idx + 1) % len(self.key_pool) logger.info(f"[SemanticScholar] Rotated API key (active index: {self.current_key_idx})") def get_headers(self) -> Dict[str, str]: """Builds outbound headers, strictly enforcing 'x-api-key' when available.""" headers = { "User-Agent": cfg.USER_AGENT, "Accept": "application/json", } api_key = self._get_api_key() if api_key: headers["x-api-key"] = api_key return headers def verify_doi(self, doi: str) -> Optional[Dict[str, Any]]: """ Verifies citation metadata for a DOI against Semantic Scholar. Applies token-bucket pacing on every attempt and properly formats the DOI prefix. """ if not doi or self.circuit_breaker.is_tripped(): return None clean_doi = sanitize_doi(doi) if not clean_doi: return None # Prefix with DOI: and encode with safe='' to avoid route path fragmentation encoded_id = f"DOI:{urllib.parse.quote(clean_doi, safe='')}" url = f"{self.BASE_URL}/paper/{encoded_id}" params = {"fields": "title,venue,year,externalIds,openAccessPdf"} def _do_request(): # Paced inside the execution block so retries

observe the 1 RPS limit self.limiter.acquire() headers = self.get_headers() return self.session.get(url, params=params, headers=headers, timeout=cfg.DEFAULT_TIMEOUT) try: resp = execute_with_429_retry(request_fn=_do_request, max_retries=cfg.MAX_RETRIES, initial_backoff=2.0, service_name="Semantic Scholar") if resp is None: self.last_status_code = None self.circuit_breaker.record_failure(None, "Timeout or connection error") return None self.last_status_code = resp.status_code if resp.status_code == 200: self.circuit_breaker.record_success() data = resp.json() venue = data.get("venue") or "" oa = data.get("openAccessPdf") or {} return { "confirmed": True, "title": data.get("title", ""), "venue": venue, "year": str(data.get("year") or ""), "pdf_link": oa.get("url", ""), "external_ids": data.get("externalIds", {}) } elif resp.status_code == 404: # 404 indicates the paper is unindexed in S2, not an infrastructure failure return None elif resp.status_code == 429: self._rotate_key() if self.on_429_callback: try: self.on_429_callback() except Exception: pass self.circuit_breaker.record_failure(429, "Rate limited") return None elif resp.status_code in (401, 403): self._rotate_key() self.circuit_breaker.record_failure(resp.status_code, resp.text[:100]) return None return None except Exception as e: logger.warning(f"[SemanticScholar] verify_doi failed for '{doi}': {e}") self.circuit_breaker.record_failure(None, str(e)) return None def search_papers(self, query: str, offset: int = 0, limit: int = 50, year_range: Optional[str] = None) -> Tuple[List[Dict[str, Any]], int, bool]: """ Searches Semantic Scholar with pagination parameters. Returns: (paper_records, total_count, has_more) """ if self.circuit_breaker.is_tripped() or not query: return [], 0, False url = f"{self.BASE_URL}/paper/search" limit = min(limit, 50) params: Dict[str, Any] = { "query": query, "offset": offset, "limit": limit, "fields": "title,authors,year,venue,externalIds,abstract,openAccessPdf" } if year_range and year_range.lower() != "all years": parts = [p.strip() for p in year_range.split("-") if p.strip().isdigit()] if len(parts) == 2: params["year"] = f"{parts[0]}-{parts[1]}" elif len(parts) == 1: params["year"] = parts[0] def _do_request(): # Paced inside retry closure so automatic backoffs do not bypass rate limits self.limiter.acquire() headers = self.get_headers() return self.session.get(url, params=params, headers=headers, timeout=cfg.DEFAULT_TIMEOUT) try: resp = execute_with_429_retry(request_fn=_do_request, max_retries=cfg.MAX_RETRIES, initial_backoff=2.0, service_name="Semantic Scholar Search") if resp is None: self.last_status_code = None self.circuit_breaker.record_failure(None, "Search request timeout or error") return [], 0, False self.last_status_code = resp.status_code if resp.status_code == 200: self.circuit_breaker.record_success() data = resp.json() total = data.get("total", 0) items = data.get("data", []) has_more = (offset + len(items)) < total and len(items) > 0 return items, total, has_more elif resp.status_code == 429: self._rotate_key() self.circuit_breaker.record_failure(429, "Search rate limited") return [], 0, False elif resp.status_code in (401, 403): self._rotate_key() self.circuit_breaker.record_failure(resp.status_code, resp.text[:100]) return [], 0, False return [], 0, False except Exception as e: logger.error(f"[SemanticScholar] search_papers failed: {e}") self.circuit_breaker.record_failure(None, str(e)) return [], 0, False def verify_title(self, title: str) -> Optional[Dict[str, Any]]: """Verifies metadata for a title against Semantic Scholar Graph API.""" if not title or self.circuit_breaker.is_tripped(): return None items, _, _ = self.search_papers(query=title, limit=1) if not items: return None data = items[0] matched_title = data.get("title", "") s1 = clean_matching_title(title) s2 = clean_matching_title(matched_title) if SequenceMatcher(None, s1, s2).ratio() < 0.65: return None venue = data.get("venue") or "" oa = data.get("openAccessPdf") or {} ext_ids = data.get("externalIds") or {} doi = sanitize_doi(ext_ids.get("DOI", "")) return { "confirmed": True, "title": matched_title, "venue": venue, "year": str(data.get("year") or ""), "doi": doi, "pdf_link": oa.get("url", ""), "external_ids": ext_ids } What is the error here

Response: The primary error is that **`execute_with_429_retry` handles HTTP 429 internally, but the key rotation and circuit breaker failure recording are located outside of it**. This makes key rotation and backoff handling ineffective.

1. `_rotate_key()` and `record_failure(429)` Never Run During Retries

`execute_with_429_retry` intercepts status code 429 inside its loop and retries `_do_request`.

- Because `_rotate_key()` is placed **after** `execute_with_429_retry` finishes:

```
resp = execute_with_429_retry(...)
# ...
elif resp.status_code == 429:
    self._rotate_key()  # <--- Only reached AFTER all retries fail!
```

- Every retry within `execute_with_429_retry` re-executes `_do_request()` using the **same rate-limited API key** that caused the 429 in the first place, burning through all retries before rotating.
- Key rotation should occur **inside** `_do_request` or via a retry callback whenever a 429 or 401/403 status is received.

2. Double-Wait / Lock Contention inside the Retry Loop

In both `verify_doi` and `search_papers`:

```
def _do_request():
    self.limiter.acquire()
    headers = self.get_headers()
    return self.session.get(...)
```

When `execute_with_429_retry` detects a 429 response, it applies its own exponential sleep (e.g., 2s, 4s, 8s). Once that backoff timer expires, `_do_request()` immediately forces an additional `self.limiter.acquire()` wait (1.05s). While safe, combining external retry sleep with an internal token bucket acquisition can cause cascading timeouts against `DEFAULT_TIMEOUT` when under load.

3. DOI Prefix Formatting in `verify_doi`

Semantic Scholar's Paper lookup endpoint expects:

```
https://api.semanticscholar.org/graph/v1/paper/{paper_id}
```

Where `{paper_id}` can be `DOI:<doi>`. However, passing `safe=''` encodes forward slashes (`/` → `%2F`):

```
encoded_id = f"DOI:{urllib.parse.quote(clean_doi, safe='')}"
```

While standard URL encoding encodes slashes, Semantic Scholar's routing proxy frequently expects standard path formatting or `safe='/'` for DOIs (e.g., `DOI:10.1038/nature12373`, not `DOI:10.1038%2Fnature12373`), which can lead to intermittent `400 Bad Request` or `404 Not Found` responses.

4. `clean_matching_title` Title Verification Bias

In `verify_title`:

```
s1 = clean_matching_title(title)
s2 = clean_matching_title(matched_title)
if SequenceMatcher(None, s1, s2).ratio() < 0.65:
    return None
```

`clean_matching_title` is imported inside `verify_doi`'s file header, but unlike the OpenAlex client, if `title` is short or contains punctuation/subtitles stripped unequally, `0.65` can produce false negatives. More importantly, Semantic Scholar search results return query relevance ranked, but `clean_title` isn't sanitized of quotes or boolean search tokens before being passed into `/paper/search`.

Corrected Implementation Pattern

Pass a retry callback or handle rotation inside the retry mechanism:

```
def _do_request():
    self.limiter.acquire()
    headers = self.get_headers()
    resp = self.session.get(url, params=params, headers=headers, timeout=cfg.DEFAULT_TIMEOUT)
    if resp.status_code in (429, 401, 403):
        self._rotate_key()  # Rotates immediately so the next retry uses a fresh key
    return resp
```

User prompt: How should execute_with_429_retry be implemented to properly support token rotation on 429?

Response: To properly support API key rotation alongside HTTP 429 handling, `execute_with_429_retry` must incorporate an **on-retry callback** (or hook).

When a 429 occurs, the handler should:

1. Parse standard rate-limiting response headers (`Retry-After`).
2. Invoke the rotation callback *before* entering backoff or before the next attempt, ensuring subsequent calls use the next available credentials.
3. Fall back to exponential backoff with jitter if no `Retry-After` header is supplied.

Implementation: `utils/retry_handlers.py`

```
"""
utils/retry_handlers.py - Robust HTTP retry handler with rate limit awareness and credential rotation.
"""

import time
import random
import logging
from typing import Callable, Optional, Any
import requests

logger = logging.getLogger("RetryHandler")


def execute_with_429_retry(
    request_fn: Callable[[], requests.Response],
    max_retries: int = 3,
    initial_backoff: float = 1.0,
    backoff_factor: float = 2.0,
    service_name: str = "External Service",
    on_retry_callback: Optional[Callable[[requests.Response, int], None]] = None,
) -> Optional[requests.Response]:
    """
    Executes an HTTP request function with retry handling on HTTP 429 and transient 5xx errors.

    Args:
        request_fn: Zero-argument callable making the network request.
        max_retries: Total number of retry attempts before giving up.
        initial_backoff: Base sleep duration in seconds.
        backoff_factor: Multiplier applied per retry cycle.
        service_name: Label used in log statements.
        on_retry_callback: Hook invoked on failure: fn(response, attempt_number).
                           Use this to rotate keys or notify circuit breakers.
```

```
    Returns:
        The final requests.Response object, or None if retries fail/exhaust.
    """
    attempt = 0
    current_backoff = initial_backoff

    while attempt <= max_retries:
        resp = None
        try:
            resp = request_fn()
        except requests.RequestException as exc:
            logger.warning(
                f"[{service_name}] Request error on attempt {attempt + 1}/{max_retries + 1}: {exc}"
            )
            if attempt == max_retries:
                return None
            sleep_duration = current_backoff + random.uniform(0.1, 0.5)
            time.sleep(sleep_duration)
            current_backoff *= backoff_factor
            attempt += 1
            continue

        # Successful or acceptable client response
        if resp.status_code not in (429, 502, 503, 504):
            return resp

        # Exhausted attempts
        if attempt == max_retries:
            logger.error(
                f"[{service_name}] Max retries reached ({max_retries}). Final status: {resp.status_code}"
            )
            return resp

        # Trigger credential rotation or state updates
        if on_retry_callback:
            try:
                on_retry_callback(resp, attempt)
            except Exception as cb_err:
                logger.error(f"[{service_name}] Error during retry callback: {cb_err}")

        # Determine sleep duration: honor Retry-After header if returned
        retry_after = resp.headers.get("Retry-After")
        if retry_after:
            try:
                wait_seconds = float(retry_after)
            except ValueError:
                wait_seconds = current_backoff
        else:
            # Full jitter exponential backoff: Uniform(0, current_backoff)
            wait_seconds = random.uniform(0.5, current_backoff)
            current_backoff *= backoff_factor

        logger.warning(
            f"[{service_name}] Received HTTP {resp.status_code}. Retrying in {wait_seconds:.2f}s "
            f"(attempt {attempt + 1}/{max_retries})..."
        )
        time.sleep(wait_seconds)
        attempt += 1

    return resp
```

How to Hook It Up in the Client

In your client (such as `SemanticScholarClient` or `OpenAlexClient`), pass `self._on_retry` into `execute_with_429_retry`:

```
class SemanticScholarClient:
    def _on_retry(self, resp: requests.Response, attempt: int):
        if resp.status_code in (429, 401, 403):
            # Advance key pointer so the next request_fn() call picks up a new key
            self._rotate_key()

    def verify_doi(self, doi: str):
        # ...
        def _do_request():
            self.limiter.acquire()
            headers = self.get_headers()  # Fetches current active rotated key
            return self.session.get(url, params=params, headers=headers, timeout=cfg.DEFAULT_TIMEOUT)

        resp = execute_with_429_retry(
```

```
            request_fn=_do_request,
            max_retries=len(self.key_pool) if self.key_pool else cfg.MAX_RETRIES,
            initial_backoff=2.0,
            service_name="Semantic Scholar",
            on_retry_callback=self._on_retry
        )
        return resp
```

User prompt: Please rewrite the full SemanticScholarClient code using the new on_retry_callback pattern.

Response: Here is the full, updated `SemanticScholarClient` incorporating the `on_retry_callback` pattern, immediate key rotation on 429/401/403, and standardized URL-safe formatting for DOI queries.

```
"""
services/academic/semantic_scholar.py - Semantic Scholar Graph API Client with Strict Rate Limiting.
Unified Academic Paper Harvester, Verifier & Notification Pipeline.

Requirements:
- Strict rate limit of 1 request per second (cumulative across all endpoints) with 1.05s safety buffer.
- Exponential backoff retry handler catching HTTP 429 (Too Many Requests).
- Outbound headers must include 'x-api-key': os.getenv('S2_API_KEY') on every outbound request.
- Token pooling and rotation when multiple keys are provided via on_retry_callback.
"""

import os
import urllib.parse
import logging
from typing import Optional, Dict, Any, List, Tuple
from difflib import SequenceMatcher
import requests

from config import cfg
from utils.rate_limiters import GLOBAL_SEMANTIC_SCHOLAR_LIMITER
from utils.retry_handlers import execute_with_429_retry
from utils.circuit_breaker import CircuitBreaker
from harvester.models import clean_matching_title, sanitize_doi

logger = logging.getLogger("SemanticScholarClient")

_S2_RATE_LIMITER = GLOBAL_SEMANTIC_SCHOLAR_LIMITER


class SemanticScholarClient:
    """
    Client for Semantic Scholar Graph API.
    Enforces strict 1 req/sec pacing, HTTP 429 exponential backoff with key rotation callbacks,
    and mandatory 'x-api-key' header forwarding.
    """

    BASE_URL = "https://api.semanticscholar.org/graph/v1"

    def __init__(self, circuit_breaker: Optional[CircuitBreaker] = None):
        self.circuit_breaker = circuit_breaker or CircuitBreaker("Semantic Scholar API", failure_threshold=3)
        self.session = requests.Session()
        self.limiter = _S2_RATE_LIMITER
        self.key_pool: List[str] = getattr(cfg, "get_key_pool", lambda service: [])("semantic_scholar")
        self.current_key_idx: int = 0
        self.last_status_code: Optional[int] = None
        self.on_429_callback: Optional[Any] = None

    def _get_api_key(self) -> str:
        """Retrieves active S2 API key from key pool first, falling back to env/config."""
        if self.key_pool:
            return self.key_pool[self.current_key_idx % len(self.key_pool)].strip()

        env_key = (
            os.getenv("S2_API_KEY")
            or getattr(cfg, "S2_API_KEY", None)
            or getattr(cfg, "SEMANTIC_SCHOLAR_API_KEY", None)
            or ""
        )
        return env_key.strip()

    def _rotate_key(self) -> None:
        """Rotates to next key in pool upon receiving rate limit or auth response."""
        if self.key_pool and len(self.key_pool) > 1:
            self.current_key_idx = (self.current_key_idx + 1) % len(self.key_pool)
            logger.info(f"[SemanticScholar] Rotated API key (active index: {self.current_key_idx})")
```

```
    def _handle_retry_callback(self, resp: requests.Response, attempt: int) -> None:
        """
        Callback passed to execute_with_429_retry.
        Fires on HTTP 429 or auth errors to rotate credentials before the next attempt.
        """
        if resp.status_code in (429, 401, 403):
            self._rotate_key()

        if resp.status_code == 429 and self.on_429_callback:
            try:
                self.on_429_callback()
            except Exception as e:
                logger.warning(f"[SemanticScholar] on_429_callback raised an exception: {e}")

    def get_headers(self) -> Dict[str, str]:
        """Builds outbound headers, forwarding 'x-api-key' dynamically from the rotated pool."""
        headers = {
            "User-Agent": getattr(cfg, "USER_AGENT", "AcademicHarvester/4.0"),
            "Accept": "application/json",
        }
        api_key = self._get_api_key()
        if api_key:
            headers["x-api-key"] = api_key
        return headers

    def verify_doi(self, doi: str) -> Optional[Dict[str, Any]]:
        """
        Verifies citation metadata for a DOI against Semantic Scholar.
        Applies token-bucket pacing on every attempt and properly formats the DOI prefix.
        """
        if not doi or self.circuit_breaker.is_tripped():
            return None

        clean_doi = sanitize_doi(doi)
        if not clean_doi:
            return None

        # Keep forward slashes unencoded so path-based DOI routing resolves correctly
        encoded_id = f"DOI:{urllib.parse.quote(clean_doi, safe='/')}"
        url = f"{self.BASE_URL}/paper/{encoded_id}"
        params = {"fields": "title,venue,year,externalIds,openAccessPdf"}

        def _do_request() -> requests.Response:
            self.limiter.acquire()
            headers = self.get_headers()
            return self.session.get(url, params=params, headers=headers, timeout=getattr(cfg, "DEFAULT_TIMEOUT", 10)

        try:
            # Scale retries up to key pool size if multiple keys are available
            max_retries = max(getattr(cfg, "MAX_RETRIES", 3), len(self.key_pool))

            resp = execute_with_429_retry(
                request_fn=_do_request,
                max_retries=max_retries,
                initial_backoff=2.0,
                service_name="Semantic Scholar",
                on_retry_callback=self._handle_retry_callback
            )

            if resp is None:
                self.last_status_code = None
                self.circuit_breaker.record_failure(None, "Timeout or connection error")
                return None

            self.last_status_code = resp.status_code

            if resp.status_code == 200:
                self.circuit_breaker.record_success()
                data = resp.json()
                venue = data.get("venue") or ""
                oa = data.get("openAccessPdf") or {}
                return {
                    "confirmed": True,
                    "title": data.get("title", ""),
                    "venue": venue,
                    "year": str(data.get("year") or ""),
                    "pdf_link": oa.get("url", ""),
                    "external_ids": data.get("externalIds", {})
```

```
                }
            elif resp.status_code == 404:
                return None
            elif resp.status_code == 429:
                self.circuit_breaker.record_failure(429, "Rate limited across all keys")
                return None
            elif resp.status_code in (401, 403):
                self.circuit_breaker.record_failure(resp.status_code, resp.text[:100])
                return None

            return None

        except Exception as e:
            logger.warning(f"[SemanticScholar] verify_doi failed for '{doi}': {e}")
            self.circuit_breaker.record_failure(None, str(e))
            return None

    def search_papers(
        self,
        query: str,
        offset: int = 0,
        limit: int = 50,
        year_range: Optional[str] = None
    ) -> Tuple[List[Dict[str, Any]], int, bool]:
        """
        Searches Semantic Scholar with pagination parameters and automatic key rotation on 429.
        Returns: (paper_records, total_count, has_more)
        """
        if self.circuit_breaker.is_tripped() or not query:
            return [], 0, False

        url = f"{self.BASE_URL}/paper/search"
        limit = min(limit, 50)
        params: Dict[str, Any] = {
            "query": query,
            "offset": offset,
            "limit": limit,
            "fields": "title,authors,year,venue,externalIds,abstract,openAccessPdf"
        }

        if year_range and year_range.lower() != "all years":
            parts = [p.strip() for p in year_range.split("-") if p.strip().isdigit()]
            if len(parts) == 2:
                params["year"] = f"{parts[0]}-{parts[1]}"
            elif len(parts) == 1:
                params["year"] = parts[0]

        def _do_request() -> requests.Response:
            self.limiter.acquire()
            headers = self.get_headers()
            return self.session.get(url, params=params, headers=headers, timeout=getattr(cfg, "DEFAULT_TIMEOUT", 10)

        try:
            max_retries = max(getattr(cfg, "MAX_RETRIES", 3), len(self.key_pool))

            resp = execute_with_429_retry(
                request_fn=_do_request,
                max_retries=max_retries,
                initial_backoff=2.0,
                service_name="Semantic Scholar Search",
                on_retry_callback=self._handle_retry_callback
            )

            if resp is None:
                self.last_status_code = None
                self.circuit_breaker.record_failure(None, "Search request timeout or error")
                return [], 0, False

            self.last_status_code = resp.status_code

            if resp.status_code == 200:
                self.circuit_breaker.record_success()
                data = resp.json()
                total = data.get("total", 0)
                items = data.get("data", [])
                has_more = (offset + len(items)) < total and len(items) > 0
                return items, total, has_more
            elif resp.status_code == 429:
                self.circuit_breaker.record_failure(429, "Search rate limited across all keys")
```

```
                return [], 0, False
            elif resp.status_code in (401, 403):
                self.circuit_breaker.record_failure(resp.status_code, resp.text[:100])
                return [], 0, False

            return [], 0, False

        except Exception as e:
            logger.error(f"[SemanticScholar] search_papers failed: {e}")
            self.circuit_breaker.record_failure(None, str(e))
            return [], 0, False

    def verify_title(self, title: str) -> Optional[Dict[str, Any]]:
        """Verifies metadata for a title against Semantic Scholar Graph API."""
        if not title or self.circuit_breaker.is_tripped():
            return None

        items, _, _ = self.search_papers(query=title, limit=1)
        if not items:
            return None

        data = items[0]
        matched_title = data.get("title", "")

        s1 = clean_matching_title(title)
        s2 = clean_matching_title(matched_title)
        if SequenceMatcher(None, s1, s2).ratio() < 0.65:
            return None

        venue = data.get("venue") or ""
        oa = data.get("openAccessPdf") or {}
        ext_ids = data.get("externalIds") or {}
        doi = sanitize_doi(ext_ids.get("DOI", ""))

        return {
            "confirmed": True,
            "title": matched_title,
            "venue": venue,
            "year": str(data.get("year") or ""),
            "doi": doi,
            "pdf_link": oa.get("url", ""),
            "external_ids": ext_ids
        }
```

---

*User prompt: """ utils/retry_handlers.py - Exponential backoff retry handlers for HTTP 429 and transient errors. Unified Academic Paper Harvester, Verifier & Notification Pipeline. """ import time import random import logging from typing import Callable, Any, Optional import requests from tenacity import (retry, stop_after_attempt, wait_exponential, retry_if_exception_type, before_sleep_log) logger = logging.getLogger("RetryHandler") def execute_with_429_retry(request_fn: Callable[[], requests.Response], max_retries: int = 4, initial_backoff: float = 1.5, max_backoff: float = 30.0, service_name: str = "API") -> Optional[requests.Response]: """ Executes an HTTP request function with custom exponential backoff retry specifically tuned for HTTP 429 (Too Many Requests) and 5xx errors. Extracts and honors 'Retry-After' headers when provided by the server. """ for attempt in range(1, max_retries + 1): try: resp = request_fn() if resp.status_code == 429: # Check for Retry-After header retry_after_str = resp.headers.get("Retry-After") if retry_after_str: try: sleep_time = float(retry_after_str) except ValueError: sleep_time = min(max_backoff, initial_backoff * (2 ** (attempt - 1))) else: # Exponential backoff with small random jitter sleep_time = min(max_backoff, initial_backoff * (2 ** (attempt - 1)) + random.uniform(0.1, 0.5)) if attempt < max_retries: logger.warning(f"⚠️ [{service_name}] HTTP 429 (Rate Limit Hit). Attempt {attempt}/{max_retries}. " f"Backing off for {sleep_time:.2f}s...") time.sleep(sleep_time) continue else: logger.error(f"[{service_name}] HTTP 429 persistence after {max_retries} attempts.") return resp return resp except (requests.exceptions.ConnectionError, requests.exceptions.Timeout) as e: if attempt < max_retries: sleep_time = min(max_backoff, initial_backoff * (2 ** (attempt - 1))) logger.warning(f"[{service_name}] Network error ({e}). Retrying in {sleep_time:.2f}s (Attempt {attempt}/{max_retries})...") time.sleep(sleep_time) else: logger.error(f"[{service_name}] Request failed after {max_retries} attempts: {e}") raise e return None """ utils/retry_handlers.py - Robust HTTP retry handler with rate limit awareness and credential rotation. """ import time import random import logging from typing import Callable, Optional, Any import requests logger = logging.getLogger("RetryHandler") def execute_with_429_retry(request_fn: Callable[[], requests.Response], max_retries: int = 3, initial_backoff: float = 1.0, backoff_factor: float = 2.0, service_name: str = "External Service", on_retry_callback: Optional[Callable[[requests.Response, int], None]] = None,) -> Optional[requests.Response]: """ Executes an HTTP request function with retry handling on HTTP 429 and transient 5xx errors. Args: request_fn: Zero-argument callable making the network request. max_retries: Total number of retry attempts before giving up. initial_backoff: Base sleep duration in seconds. backoff_factor: Multiplier applied per retry cycle. service_name: Label used in log statements. on_retry_callback: Hook invoked on failure: fn(response, attempt_number). Use this to rotate keys or notify circuit breakers. Returns: The final requests.Response object, or None if retries fail/exhaust. """ attempt = 0 current_backoff = initial_backoff while attempt <= max_retries: resp = None try: resp = request_fn() except requests.RequestException as exc: logger.warning(f"[{service_name}] Request error on attempt {attempt + 1}/{max_retries + 1}: {exc}") if attempt == max_retries: return None sleep_duration = current_backoff + random.uniform(0.1, 0.5) time.sleep(sleep_duration) current_backoff *= backoff_factor attempt += 1 continue # Successful or acceptable client response if resp.status_code not in (429, 502, 503, 504): return resp # Exhausted attempts if attempt == max_retries: logger.error(f"[{service_name}] Max retries reached ({max_retries}). Final status: {resp.status_code}") return resp # Trigger credential rotation or state updates if on_retry_callback: try: on_retry_callback(resp, attempt) except Exception as cb_err: logger.error(f"[{service_name}] Error during retry callback: {cb_err}") # Determine sleep duration: honor Retry-After header if returned retry_after = resp.headers.get("Retry-After")*

*if retry_after: try: wait_seconds = float(retry_after) except ValueError: wait_seconds = current_backoff else: # Full jitter exponential backoff: Uniform(0, current_backoff) wait_seconds = random.uniform(0.5, current_backoff) current_backoff *= backoff_factor logger.warning(f"[{service_name}] Received HTTP {resp.status_code}. Retrying in {wait_seconds:.2f}s " f"(attempt {attempt + 1}/{max_retries})...") time.sleep(wait_seconds) attempt += 1 return resp*
Merge this code and resend pls ensure that it follows all correctly

Response: Here is the consolidated, production-ready version of `utils/retry_handlers.py`. It combines the best elements of both implementations: explicit handling of `Retry-After` headers (supporting both integer seconds and HTTP dates), customizable maximum backoff capping, credential/token rotation callbacks, and graceful network error recovery.

```
"""
utils/retry_handlers.py - Robust HTTP retry handler with rate limit awareness and credential rotation.
Unified Academic Paper Harvester, Verifier & Notification Pipeline.
"""

import time
import random
import logging
from email.utils import parsedate_to_datetime
from typing import Callable, Optional
import requests

logger = logging.getLogger("RetryHandler")


def _parse_retry_after(header_val: Optional[str]) -> Optional[float]:
    """Parses a Retry-After header which may be integer seconds or an HTTP date string."""
    if not header_val:
        return None
    header_val = header_val.strip()
    try:
        return max(0.0, float(header_val))
    except ValueError:
        pass
    try:
        target_dt = parsedate_to_datetime(header_val)
        delay = (target_dt.timestamp() - time.time())
        return max(0.0, delay)
    except Exception:
        return None


def execute_with_429_retry(
    request_fn: Callable[[], requests.Response],
    max_retries: int = 4,
    initial_backoff: float = 1.5,
    backoff_factor: float = 2.0,
    max_backoff: float = 60.0,
    service_name: str = "External Service",
    on_retry_callback: Optional[Callable[[requests.Response, int], None]] = None,
) -> Optional[requests.Response]:
    """
    Executes an HTTP request function with retry handling on HTTP 429, transient 5xx errors,
    and network dropouts. Supports token rotation callbacks and honors Retry-After headers.

    Args:
        request_fn: Zero-argument callable executing the HTTP request.
        max_retries: Total number of retry attempts before giving up.
        initial_backoff: Initial sleep duration in seconds.
        backoff_factor: Exponential multiplier applied after each retry.
        max_backoff: Upper bound for any single backoff wait duration.
        service_name: Label used for contextual log messages.
        on_retry_callback: Hook triggered on retryable response: fn(response, attempt).
                           Used to rotate API keys or update rate limiter state.

    Returns:
        The final requests.Response object, or None if retries are exhausted by exceptions.
    """
    attempt = 0
    current_backoff = initial_backoff

    while attempt <= max_retries:
        resp: Optional[requests.Response] = None
        try:
            resp = request_fn()
        except (requests.exceptions.ConnectionError, requests.exceptions.Timeout) as exc:
            if attempt == max_retries:
                logger.error(
                    f"[{service_name}] Request failed after {max_retries} retries due to network error: {exc}"
                )
```

```
                return None

            sleep_duration = min(max_backoff, current_backoff + random.uniform(0.1, 0.5))
            logger.warning(
                f"[{service_name}] Network error ({exc.__class__.__name__}). Retrying in {sleep_duration:.2f}s "
                f"(attempt {attempt + 1}/{max_retries})..."
            )
            time.sleep(sleep_duration)
            current_backoff = min(max_backoff, current_backoff * backoff_factor)
            attempt += 1
            continue
        except requests.RequestException as exc:
            logger.error(f"[{service_name}] Non-retryable request exception: {exc}")
            return None

        # Success or non-retryable client error
        if resp.status_code not in (429, 500, 502, 503, 504):
            return resp

        # Max retries reached
        if attempt == max_retries:
            logger.error(
                f"[{service_name}] Max retries reached ({max_retries}). Final status: {resp.status_code}"
            )
            return resp

        # Fire credential rotation or monitoring callback
        if on_retry_callback:
            try:
                on_retry_callback(resp, attempt)
            except Exception as cb_err:
                logger.warning(f"[{service_name}] Error during on_retry_callback: {cb_err}")

        # Compute delay: prioritize server-provided Retry-After over standard backoff
        server_retry_after = _parse_retry_after(resp.headers.get("Retry-After"))
        if server_retry_after is not None:
            wait_seconds = min(max_backoff, server_retry_after)
        else:
            jitter = random.uniform(0.1, 0.5)
            wait_seconds = min(max_backoff, current_backoff + jitter)
            current_backoff = min(max_backoff, current_backoff * backoff_factor)

        logger.warning(
            f"⚠️ [{service_name}] HTTP {resp.status_code}. Retrying in {wait_seconds:.2f}s "
            f"(attempt {attempt + 1}/{max_retries})..."
        )
        time.sleep(wait_seconds)
        attempt += 1

    return resp
```
